

작은 모델이 지휘한다

라우팅 · 사다리 · 상태 분리 · 예산

모든 요청에 최상위 모델을 쓰지 않는다

실패가 반복되면 그때 올린다

작업별 상태와 비용을 코드로 관리한다

청구서의 구조

출력은 입력의 2~8 배

캐시된 입력과 비교하면 최대 50 배까지 벌어진다

사고 토큰은 출력 단가로 청구된다

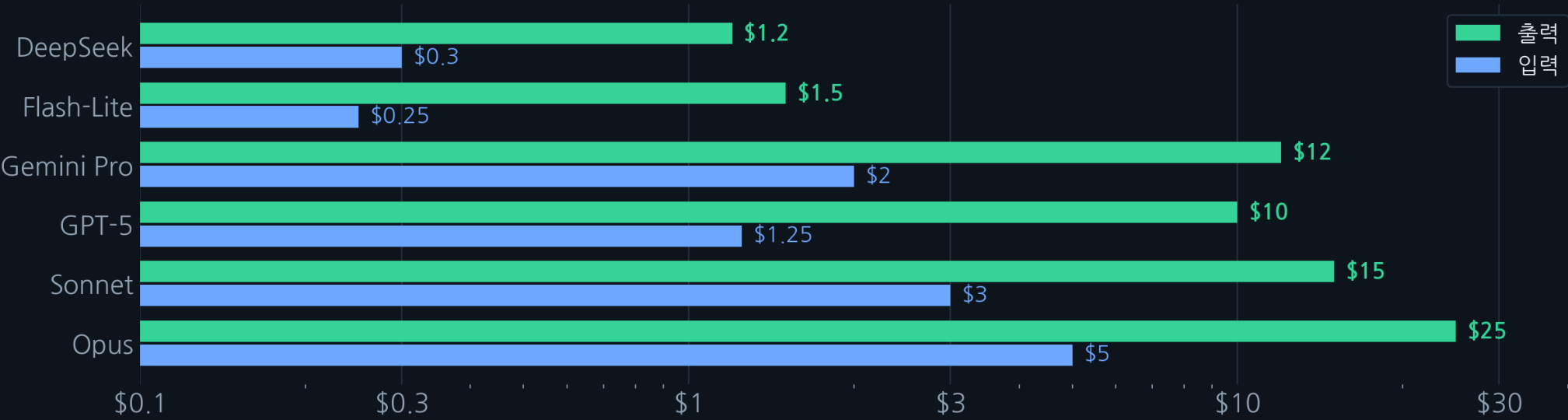
그래서 '덜 쓰기' 보다 '다시 안 보내기' 가 크다

같은 토큰 , 다른 단가

출력은 입력의 4~8 배

비싼 쪽 (출력) 을 줄이는 습관이 먼저다
캐시 읽기는 입력의 0.1 배 — 캐시가 있는 이유

모델별 입력·출력 단가 (USD / 100만 토큰, 로그 축)



단가 표로 다시 보기

출력 ÷ 입력 4~8 배 , 캐시 읽기 0.02~0.1 배

단가는 2026-09-20 공개 리스트 스냅샷 (USD / 100 만 토큰)

라이브 조회값이 아니라 저장된 표라서 , 갱신은 한 곳에서 한다

모델	입력 \$/1M	출력 \$/1M	출력 ÷ 입력	캐시 읽기
Claude Opus 4.6	\$5	\$25	5 배	0.1 배
Claude Sonnet 4.6	\$3	\$15	5 배	0.1 배
GPT-5	\$1.25	\$10	8 배	0.1 배
Gemini 3.1 Pro Preview (≤200k)	\$2	\$12	6 배	0.1 배
Gemini 3.1 Flash-Lite	\$0.25	\$1.5	6 배	0.1 배
DeepSeek V4.1 Flash (peak)	\$0.3	\$1.2	4 배	0.02 배

긴 세션은 컴팩션으로 접는다

\$2.32 → \$0.93 (60% 절감)

10 턴마다 접으면 \$1.23 (47% 절감)

너무 자주 접으면 요약 호출이 이득을 먹는다

최적 주기는 세션 길이와 요약 비용으로 정해진다

툴 출력과 서버에이전트는 전체 비용으로 본다

\$5.6010 → \$2.7360

툴 읽기 누적 전송 1,680,000 토큰 (20 턴)

컴팩션 \$3.4425 · 서버에이전트 \$2.736

자식 호출 · 요약 비용까지 포함해 비교한다

무엇을 자동화하는가

요청마다 사람이 모델을 고르지 않는다

설계 · 구현 · 설명 · 정형 작업을 분류해 티어를 고른다

부수 단어 (파일 업로드 · 검색) 보다 작업 목적을 먼저 본다

비용 예산과 출력 상한을 함께 건다

두 정책을 섞지 않는다

라우팅 · 리워크 · 워크플로 컨트롤

라우팅 : 요청 한 건을 어느 티어로 보낼지 고른다

리워크 : 같은 일을 다시 시키는 구간을 관리한다

워크플로 컨트롤 : 앞의 둘을 포함하는 상위 개념

사다리 A · 내부 실패가 올린다

테스트가 실패하면 턴 안에서 올린다

수용 기준 (AC) 이 없으면 올릴 근거도 없다

같은 실패가 반복될수록 누적 비용이 커진다

진단과 적용을 같은 함수로 계산한다

사다리 B · 사용자 반려가 올린다

저장 발화 6 턴에서 3 회 에스컬레이션

같은 의도의 반복을 감지해 올린다

스펙 변경 · 신규 작업은 '반려'가 아니다

분류 신호가 없으면 올리지 않고 되묻는다

레버는 곱해진다

85.825% (잔여 14%)

시맨틱 캐시 → 라우팅 → 캐싱 → verbosity/effort → 배치

잔여 비용에 순차 적용한 산술 결과

레버 간 중복 · 상호작용은 검증하지 않았다

상태를 작업별로 분리한다

새 작업은 새 상태로 시작

task_id 별로 의도 이력 · 시도 횟수 · 범위를 추적한다

앞 작업의 반려 상태가 새 작업으로 새지 않는다

초기화 전 내부 호출은 명시적 오류로 처리

진단과 적용은 같은 함수로 계산한다

\$0.05875 (같은 단계 · 같은 가정)

범위 · 리셋 · 출력 상한 · 진단 / 적용을 한 곳에서 계산

두 경로가 같은 값을 내는지 테스트로 고정

티어 단가와 고정 입력 예산은 시나리오 값

입력 증가분을 두 번 더하지 않는다

4,700 을 한 번만 더한다

증가분은 직전 출력과 사용자 발화를 포함한 총량

출력 토큰을 다시 더하면 이중 계상이 된다

누적 예 : 1,419 → 6,119 → 10,819

고정 접두부를 캐시에 태운다

규약 · 시스템 프롬프트는 매 턴 그대로

툴 정의와 규약 파일을 맨 앞 고정 블록으로 모은다

변하는 값 (사용자 입력 · 타임스탬프) 은 뒤로 보낸다

적중률을 대시보드로 본다

원칙 5 · 턴 수가 비용을 지배한다

턴 2 배 → 비용 3.1 배

5 턴 \$0.26 · 20 턴 \$2.32 · 50 턴 \$12.09

안 쓰는 코드도 매 턴 다시 실행 과금된다 (YAGNI)

컨텍스트는 길이보다 '반복' 이 비쌉니다

긴 세션일수록 가파르다

5 턴 \$0.26 → 50 턴 \$12.09

20 턴에 컴팩션을 걸면 \$0.93 (60% 절감)

안 쓰는 코드도 매 턴 다시 실행 과금된다

턴 수에 따른 세션 비용 (Claude Sonnet 4.6, 고정 8,000토큰 프리픽스)

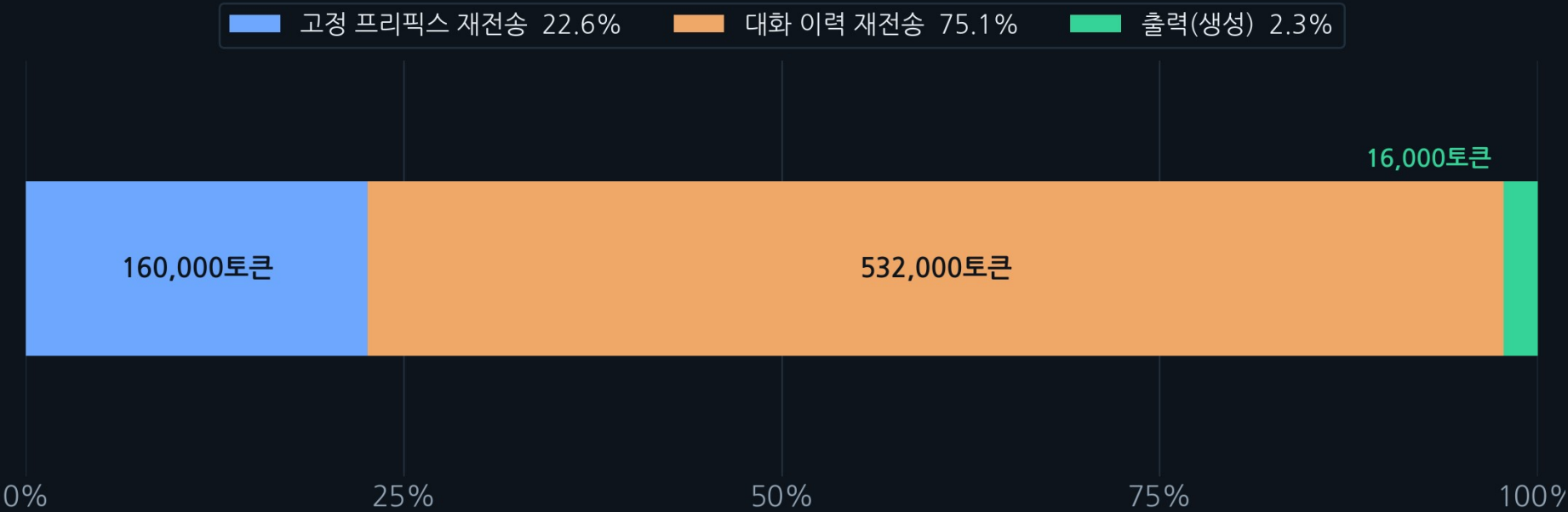


세션 토큰의 4분의 3은 이력 재전송

고정 프리픽스 22.6% · 이력 75.1%

생성 (출력) 은 2.3% 뿐이다

그래서 ' 덜 쓰기 ' 보다 ' 다시 안 보내기 ' 가 크다



캐싱 · 오늘 바로 켤 수 있는 레버

\$8.55 → \$3.22 (62.4% 절감)

쓰기 프리미엄은 두 번째 호출에서 회수된다

고정 블록 (시스템 · 톨 정의 · 규약) 을 맨 앞으로 모은다

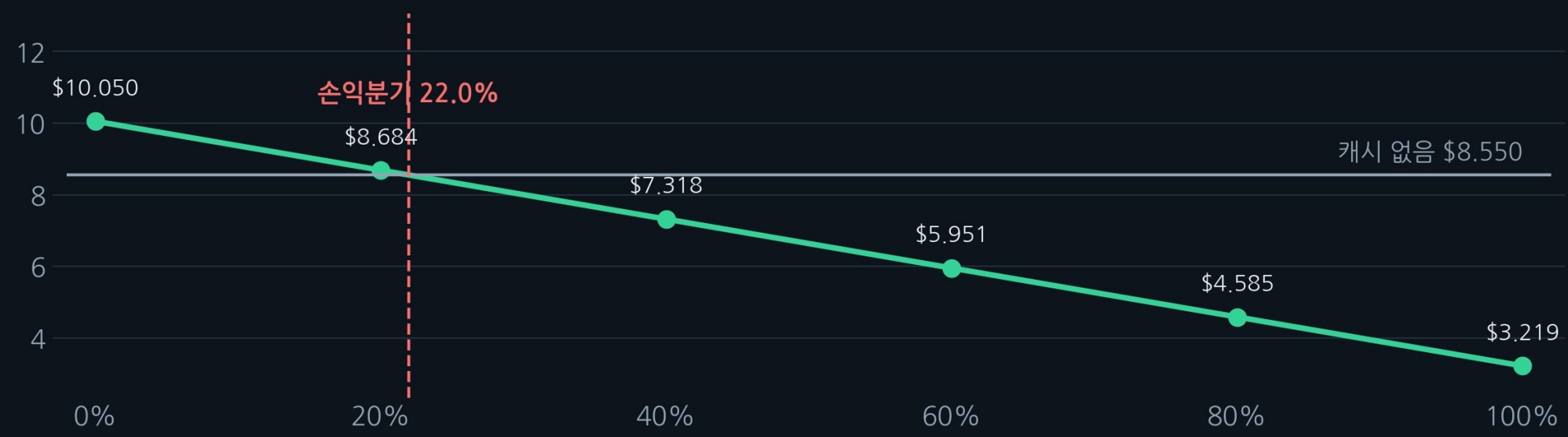
최소 길이를 확인한다 — Haiku 4.5 는 4,096 토큰

적중률 22% 면 본전

캐시 없음 \$8.550

적중률이 오를수록 단조 감소한다
미스가 섞이면 쓰기 프리미엄을 계속 문다

프롬프트 캐시 적중률에 따른 100콜 비용 (Sonnet 4.6)



웹 세션으로 넘기는 경로

구독제 경로는 증분 API 비용 0 으로 모델링

조사·정리는 웹 세션에서, 코드는 API에서

옮겨 붙일 때 수용 기준 형식을 그대로 쓴다

이 경로의 증분 종량 비용은 0 으로 모델링한다

적용 순서와 측정 루프

켜고 → 재고 → 고친다

되돌리기 쉬운 것부터 켜다 (캐시 · effort · 배치)

적중률 · reasoning 비율 · 턴 수를 주간으로 본다

절감률 목표가 아니라 품질 게이트를 먼저 정한다

마무리 · 오늘 붙일 것 하나

라우터 한 줄, 캐시 한 블록

규약 파일을 프리픽스로 고정한다

기본 티어를 한 단계 낮추고 실패 시 올린다

다음 주에 같은 작업의 비용을 다시 낸다